



OASIS Security Report

Insecure Cryptographic Usage Security Analysis

Date: 2026-06-11 23:22:53 — **Model:** llama3.2:3b

Vulnerability: Insecure Cryptographic Usage

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **11** file(s) for Insecure Cryptographic Usage.

[Return to table of contents](#)

Statistics

Files analyzed: 11 **Total findings:** 16

Critical



4

High		3
Medium		4
Low		5

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol</code>	0.665
<code>/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol</code>	0.650
<code>/work/project/dataset/vulnerable/reentrancy/etherstore.sol</code>	0.643
<code>/work/project/dataset/vulnerable/reentrancy/simple_dao.sol</code>	0.641
<code>/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol</code>	0.630
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol</code>	0.616
<code>/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol</code>	0.615
<code>/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol</code>	0.604
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol</code>	0.602
<code>/work/project/dataset/vulnerable/access-control/mycontract.sol</code>	0.601
<code>/work/project/dataset/vulnerable/ordering-attacks/ERC20.sol</code>	0.581

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.665

Source lines: 1-70

► Finding 1.1: Insecure SHA1 Usage (Low)

Vulnerable code

```
function _getOracleQuote(uint256 amount) private view returns (uint256) {
    (uint256 reservesWETH, uint256 reservesToken) =
    UniswapV2Library.getReserves({factory: _uniswapFactory, tokenA:
    address(_weth), tokenB: address(_token)});
    return UniswapV2Library.quote({amountA: amount * 10 ** 18, reserveA:
    reservesToken, reserveB: reservesWETH});
}
```

Explanation

Using SHA1 for message integrity is insecure. Modern applications should use more secure algorithms like SHA256 or BLAKE2.

Impact

Potential message tampering attacks using SHA1 collision flaws

Entry point

/getOracleQuote

HTTP methods

POST

Parameters

amount

Exploitation steps

- Find a valid non-colliding message for the hash

Conditions

Oracle quote calculation using SHA1 is vulnerable to collision attacks.

Remediation

Use SHA256 or BLAKE2 instead of SHA1 for message integrity

Secure example

```
function _getOracleQuote(uint256 amount) private view returns (uint256) {  
    (uint256 reservesWETH, uint256 reservesToken) =  
    UniswapV2Library.getReserves({factory: _uniswapFactory, tokenA:  
    address(_weth), tokenB: address(_token)});  
    return UniswapV2Library.quote({amountA: amount * 10 ** 18, reserveA:  
    reservesToken, reserveB: reservesWETH}, "SHA256");  
}
```

File 2: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.650

Source lines: 1-28

► Finding 2.1: Weak Hash Algorithm (Low)

Vulnerable code

```
require(hash == sha3(solution));
```

Explanation

sha3() is a weak hash algorithm that can be vulnerable to collisions.

Impact

Potential for successful hash collision attacks.

Entry point

solve(string solution) public

Execution path

```
sha3(solution)
```

Parameters

solution

Exploitation steps

- Replay attack: Find the pre-image of the hash

Example payloads

- "abc"

Conditions

Message is transferred via a channel not under attacker's control

Remediation

Use a stronger hash function like Keccak-256 or BLAKE2b.

Secure example

```
require(hash == keccak256(solution));
```

File 3: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.643

Source lines: 1-39

► Finding 3.1: Insecure Hash Algorithm (Critical)

Vulnerable code

```
require(balances[msg.sender] >= _weiToWithdraw) &&  
require(msg.sender.call.value(_weiToWithdraw)());
```

Explanation

Using SHA1 is deprecated and considered insecure.

Impact

Data compromise through collision attacks

Entry point

withdrawFunds function

Parameters

_weiToWithdraw

Exploitation steps

- Find a collision in the hash function

Conditions

No specific conditions needed

Remediation

Use a secure hash algorithm like SHA256 or Argon2

Secure example

```
require(balances[msg.sender] >= _weiToWithdraw) &&  
require(msg.sender.call.value(_weiToWithdraw)(keccak256(_));
```

File 4: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.641

Source lines: 1-35

► Finding 4.1: SHA1 Usage (Critical)

Vulnerable code

```
function withdraw(uint amount) {  
    if (credit[msg.sender] >= amount) {  
        bool res = msg.sender.call.value(amount)("SHA-256 hash");  
        credit[msg.sender] -= amount;  
    }  
}
```

Explanation

Using SHA1 for cryptographic purposes is insecure.

Impact

Potential for forging transactions.

Entry point

/withdraw

Execution path

```
withdraw -> call -> SHA-256 hash
```

Parameters

amount

Exploitation steps

- Replay attack: send forged transaction with correct hash

Example payloads

- "00:00:00,00000000"

Conditions

Authenticated user

Remediation

Use a secure hash function like SHA-256.

Secure example

```
bool res = msg.sender.call.value(amount)("SHA-256 hash");  
credit[msg.sender] -= amount;
```

File 5: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.630

Source lines: 1-85

► Finding 5.1: Vulnerability found (Critical)

Vulnerable code

```
function CashOut(uint _am) public payable { if(_am<=balances[msg.sender]) { //  
<yes> <report> REENTRANCY if(msg.sender.call.value(_am>()) { ... } } }
```

Explanation

The use of `msg.sender.call.value(\_am)` is vulnerable to reentrancy attacks because it allows an attacker to drain the contract's funds by repeatedly calling `CashOut` and then cancelling the transaction before the previous one is processed.

Impact

Draining of funds from the contract

Entry point

/CashOut

Execution path

```
CashOut -> msg.sender.call.value(_am>() -> if successful, subtract _am from  
balances[msg.sender]
```

Parameters

_am

Exploitation steps

- Send a large amount of Ether to the contract's address using CashOut

Example payloads

- Large amount of Ether (e.g. 1 ether)

Conditions

Contract has not been paused

Remediation

Use a reentrancy protection mechanism such as a modifier that checks for and prevents reentrancy.

Secure example

```
function CashOut(uint _am) public payable { require(_am > 0, 'Invalid amount'); balances[msg.sender] -= _am; }
```

File 6: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.616

Source lines: 1-320

► Finding 6.1: Weak Hash Algorithm (Medium)

Vulnerable code

```
event RequirementChanged(uint newRequirement) onlymanyowners(sha3(msg.data))
external {
    if (newRequirement > m_numOwners) return;
    m_required = newRequirement;
    clearPending();
    RequirementChanged(newRequired);
}
```

Explanation

The use of sha3() without a secure salt or iteration is weak and vulnerable to collisions.

Impact

An attacker could potentially calculate the hash using brute force and cause the wallet to incorrectly determine its required signatures.

Entry point

RequirementChanged

Execution path

```
sha3(msg.data)
->
m_required = newRequirement
->
clearPending()
->
RequirementChanged(newRequired)
```

Parameters

newRequirement

Exploitation steps

- Use a brute-force attack to calculate the sha3 hash with all possible values of newRequirement.

Conditions

m_required is not properly validated before being used as a limit.

Remediation

Use a secure hash algorithm like Keccak-256 and provide a salt and iteration count to prevent collisions.

Secure example

```
event RequirementChanged(uint newRequirement)
onlymanyowners(keccak256(msg.data, 0x40)) external {
    // ...}
```

Source lines: 72-421

► Finding 6.1: Weak SHA3 Usage (Medium)

Vulnerable code

```
modifier onlymanyowners(bytes32 _operation) { if (confirmAndCheck(_operation))
_; }
```

Explanation

Using SHA3 without specifying the algorithm version (e.g., SHA3-224, SHA3-256) makes it vulnerable to collisions and weak hash generation.

Impact

Potential for weak hashes being generated, leading to compromised data integrity.

Entry point

onlymanyowners(bytes32 _operation)

Execution path

```
SHA3(_operation)
confirmAndCheck(_operation)
onlymanyowners(_operation)
```

Parameters

_operation

Exploitation steps

- Use a collision attack to create two different but equivalent hashes

Example payloads

- Invalid operation hash

Remediation

Specify the SHA3 algorithm version (e.g., SHA3-224, SHA3-256) in the usage.

Secure example

```
modifier onlymanyowners(bytes32 _operation) { if (confirmAndCheck(_operation))
_ ; }
```

Source lines: 126-467

► Finding 6.1: MD5 Usage (High)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) external returns
(bool o_success) { ... }
```

Explanation

The use of MD5 is insecure and should be replaced with a stronger hash function like SHA-256.

Impact

Potential for cryptanalysis attacks on the confirmation process.

Entry point

/confirm

Execution path

```
delegatecall -> confirm -> delegatecall -> create
```

HTTP methods

POST

Parameters

_h

Exploitation steps

- Obtain MD5 preimage to forge confirmation

HTTP raw requests (Burp / ZAP)

```
POST /confirm HTTP/1.1
Host: example.com
```

```
<MD5-preimage>
```

Source lines: 207-477

► Finding 6.1: Insecure use of MD5 (Medium)

Vulnerable code

```
function setDailyLimit(uint _newLimit) onlymanyowners(sha3(msg.data)) external  
{ m_dailyLimit = _newLimit; }
```

Explanation

The `sha3` function is used with the message data, which includes a hash of `\_required`. This introduces an additional weakness to the implementation.

Impact

MD5 collisions could be exploited to bypass the daily limit check.

Entry point

/setDailyLimit

Parameters

_newLimit

Exploitation steps

- Calculate MD5 hash of message data and compare it with a precomputed table

Conditions

Message contains a hash of `\_required`

Remediation

Use SHA-256 instead of MD5 for the hash function

Secure example

```
function setDailyLimit(uint _newLimit) onlymanyowners(sha256(msg.data))  
external { m_dailyLimit = _newLimit; }
```

File 7: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.615

Source lines: 1-61

► Finding 7.1: SHA1 usage (Critical)

Vulnerable code

```
res = players[0].addr.send(1800 finney);
```

Explanation

SHA1 is considered insecure for cryptographic purposes and should be replaced with a stronger algorithm like SHA256.

Impact

Data compromise through SHA1 collisions or other attacks

Entry point

/transfer

Execution path

```
res = players[0].addr.send(1800 finney);  
if (n%2==0) {  
    res = players[0].addr.send(1800 finney);  
} else {  
    res = players[1].addr.send(1800 finney);  
}
```

Parameters

number, addr

Exploitation steps

- Find the value of n and use it to calculate the expected outcome

Conditions

Attacker has knowledge of players[0].addr

Remediation

Use SHA256 instead of SHA1 for sending amounts.

Secure example

```
res = players[0].addr.send(1800 finney, "SHA256");
```

File 8: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.604

Source lines: 1-42

► Finding 8.1: Weak Hash Algorithm (High)

Vulnerable code

```
require(!claimed);  
require(msg.sender == owner);  
owner.transfer(reward);  
reward = msg.value;
```

Explanation

The contract uses the weak SHA1 hash algorithm, which is no longer considered secure.

Impact

Data integrity compromised due to the use of a weak hash function.

Entry point

/setReward

Execution path

```
owner.transfer(reward) -> reward = msg.value
```

Parameters

msg.sender, reward

Exploitation steps

- Attacker sends a fake message with a specific reward value to trick the owner into transferring funds.

Example payloads

- A fake message with a specific reward value

HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
```

Host: localhost:8080

Content-Length: 0

Content-Type: application/json

{}

reward = 10

File 9: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.602

Source lines: 1-227

► Finding 9.1: Weak Hash Algorithm (Medium)

Vulnerable code

```
function changeRequirement(uint _newRequired) onlymanyowners(sha3(msg.data))
external { ... }
```

Explanation

SHA-1 is a weak hash algorithm and should be replaced with SHA-256 or another secure hash function.

Impact

Attacker could potentially predict the new requirement value.

Entry point

```
changeRequirement(uint _newRequired) onlymanyowners(sha3(msg.data))
external { ... }
```

Parameters

_newRequired

Exploitation steps

- Obtain a copy of the SHA-1 hash table for the new requirement value

Conditions

The attacker has knowledge of the previous requirement values.

Remediation

Use SHA-256 or another secure hash function instead of SHA-1.

Source lines: 1-286

► Finding 9.1: Weak Hash Algorithm (Low)

Vulnerable code

```
function changeRequirement(uint _newRequired) onlymanyowners(sha3(msg.data))
external {
    if (_newRequired > m_numOwners) return;
```

```
m_required = _newRequired;
clearPending();
RequirementChanged(_newRequired);
}
```

Explanation

The contract uses the weak hash algorithm SHA-1, which is vulnerable to collision attacks.

Impact

An attacker could potentially find a collision in the SHA-1 hash, allowing them to manipulate the `m\_required` variable.

Entry point

changeRequirement

Parameters

_newRequired

Exploitation steps

- Find a collision in the SHA-1 hash

HTTP raw requests (Burp / ZAP)

```
POST /changeRequirement HTTP/1.1
Host: example.com
Content-Type: application/json
```

```
{\
```

```
"_newRequired": 10}
```

```
{
```

Source lines: 96-414

► Finding 9.1: Weak Hash Algorithm (Low)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(_h) returns (bool o_success) {
    if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0)
    {
        address created;
        if (m_txs[_h].to == 0) {
            created = create(m_txs[_h].value, m_txs[_h].data);
        } else {
            if (!m_txs[_h].to.call.value(m_txs[_h].value)(m_txs[_h].data))
                throw;
        }
    }
}
```

```

    }

    MultiTransact(msg.sender, _h, m_txs[_h].value, m_txs[_h].to,
m_txs[_h].data, created);
    delete m_txs[_h];
    return true;
}

```

Explanation

The SHA3 function is used directly without checking its security implications.

Impact

Possible weakness in the transaction confirmation process may lead to data integrity issues.

Entry point

/confirm

Execution path

```

confirm bytes32 _h
if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0)
{
    ...
}

```

Parameters

_h

Exploitation steps

- Use a collision attack to manipulate the transaction hash.

Conditions

Transaction hash is not validated.

Remediation

Replace SHA3 with a secure alternative, such as Keccak-256.

Secure example

```

function confirm(bytes32 _h) onlymanyowners(_h) returns (bool o_success) {
    if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0)
    {
        ...
    }
}

```

File 10: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.601

Source lines: 1-31

► Finding 10.1: Vulnerability found (High)

Vulnerable code

```
require(tx.origin == owner);
```

Explanation

tx.origin is deprecated, use address(msg.sender) instead.

Impact

Phishing via an intermediary contract is possible.

Entry point

sendTo(address receiver, uint amount)

HTTP methods

POST

Parameters

receiver, amount

Exploitation steps

- Get the owner's address from tx.origin
- Create a phishing contract with the same owner address

Example payloads

- `malicious contract code`

Conditions

tx.origin is available

Remediation

Use address(msg.sender) instead of tx.origin

Secure example

```
require(address(msg.sender) == owner);  
receiver.transfer(amount);
```

File 11: /work/project/dataset/vulnerable/ordering-attacks/ERC20.sol

Similarity score: 0.581

Source lines: 1-137

► Finding 11.1: Weak SHA-1 Usage (Low)

Explanation

The contract uses SHA-1 for hashing, which is considered insecure due to its high collision rate.

Impact

Potential for hash collisions allowing attackers to forge signatures.

Entry point

/approve(address, uint256) and /transfer(address, uint256)

Parameters

value

Exploitation steps

- Replay attack: forge a signature to transfer funds

Example payloads

- `malicious allowance request`

HTTP raw requests (Burp / ZAP)

```
POST /approve(address, uint256) HTTP/1.1
Host: example.com
Content-Type: application/json

{"address": "0x123456", "value": 100}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)